

Prova di Algoritmi e Strutture Dati
Non si possono consultare né appunti né testi
11 Dicembre 2018 - A

Esercizio 1 Verificare o confutare applicando la definizione di ordine di grandezza:

1. $2^{n/2} \in \theta(2^n)$
2. $n \log^2(n) \in \theta(n^3)$

Esercizio 2 Calcolare la complessità in tempo dei seguenti algoritmi:

1 Test 1; 2 int $t := \log_2 n$; 3 while $t > 1$ do 4 $t := \frac{t}{2}$ •	1 Pippo($n : \text{int}$) : int 2 if $n < 2$ then 3 return 5 4 else 5 if $n < 100$ then 6 $j := 1$; while $j < 2^n$ do 7 $j := j + 1$ 8 else 9 return Pippo($\lceil \frac{n}{2} \rceil - 2$) + n •
---	---

Esercizio 3 Dato due array A e B ordinati in senso crescente e i cui elementi sono distinti, progettare un algoritmo per restituire gli elementi ripetuti (ossia che appaiono sia in A che in B) ordinati in senso decrescente. Discutere la complessità della soluzione proposta.

Esempio

Input: $A = \langle 1, 3, 5, 17 \rangle$ $B = \langle 1, 4, 5 \rangle$

Output: $C = \langle 5, 1 \rangle$

Esercizio 4 Applicando il principio di induzione, provare o confutare che $T(n) \in O(n)$, dove

$$T(n) = \begin{cases} 3 & n \leq 2 \\ T(n-1) + n & n \geq 3 \end{cases}$$

Prova di Algoritmi e Strutture Dati
Non si possono consultare né appunti né testi
11 Dicembre 2018 - B

Esercizio 1 Verificare o confutare applicando la definizione di ordine di grandezza:

- $2^{n+4} \in \theta(3^n)$
- $f(n) + g(n) \in \theta(\min(f(n), g(n)))$

Esercizio 2 Calcolare la complessità in tempo dei seguenti algoritmi:

```
1 Test 1;  int t := 0;
• 2 while t < n do
3   | t := 2t
```

```
1 Test 2;
• 2 int t := √n;
3 while t > 1 do
4   | t := t - 2
```

```
1 Pippo(n : int) : int
2 if n < 2 then
3   | return 2n
4 else
5   | int j := 1; a := 0
6   | while j ≤ n do
7     | j := j + 3
8     | for i := 1; i++; j do
9       | a := a + 1
10  | return 2 * Pippo( $\frac{n}{3}$ )
```

Esercizio 3 Dato un array A che contiene una sequenza di caratteri seguita da una sequenza di interi, progettare un algoritmo **ottimo** per trovare la posizione in cui è memorizzato l'ultimo carattere.

Esempio

Input: $A, d, E, r, 1, 8, 4, 1$

Output: 4

Esercizio 4 Risolvere la seguente equazione di ricorrenza e provarla applicando il principio di induzione:

- $$T(n) = \begin{cases} 3 & n \leq 2 \\ T(\lfloor \frac{n}{2} \rfloor) + n^2 & n \geq 3 \end{cases}$$

Prova di Algoritmi e Strutture Dati
Non si possono consultare né appunti né testi
11 Dicembre 2018 - C

Esercizio 1 Verificare o confutare applicando la definizione di ordine di grandezza:

- $\frac{n}{\log(n)} \in \theta(n^{\frac{2}{3}})$
- $2^{n+5} \in \theta(2^n)$

Esercizio 2 Calcolare la complessità in tempo dei seguenti algoritmi iterativi:

• <pre> 1 Test 1; 2 int $t := \log_4 \log_4 n$ 3 while $t > 4$ do 4 $t := \frac{t}{4}$ </pre> • <pre> 1 Test 2; 2 int $t := 1$ 3 while $t < n$ do 4 $t := t + 3$ </pre>	• <pre> 1 Pippo($n : \text{int}$) : int 2 if $n < 100$ then 3 return 1 4 else 5 $s := 0$ 6 for $i := 0; i++; \sqrt{n}$ do 7 $s := s + 3^i$ 8 while $s > 1$ do 9 $s := \frac{s}{3}$ 10 return $2 \cdot \text{Pippo}(\lfloor \frac{n}{3} \rfloor) + \text{Pippo}(\lfloor \frac{n}{5} \rfloor)$ </pre>
---	--

Esercizio 3 Dato un array A ordinato in senso non decrescente ed un intero k , progettare un algoritmo per trovare, se esiste, gli indici di una coppia di elementi la cui somma è pari a k . Discutere la complessità e l'ottimalità della soluzione.

Esempio

Input: 1, 3, 4, 5, 7 $k = 7$

Output: 2, 3 perchè $A[2] + A[3] = 7$

Esercizio 4 Risolvere la seguente equazione di ricorrenza e provarla applicando il principio di induzione:

- $$T(n) = \begin{cases} 3 & n \leq 2 \\ T(n-1) + n & n \geq 3 \end{cases}$$